

实验与配置日志：[Docker 创建新镜像]

记录人：heyeeepc

April 9, 2026

1 环境说明

在开始配置之前，当前系统环境如下：

- **OS:** centos9
 - **网络:** 虚拟网卡
-

2 准备工作

2.1 是否启动

首先，确保你的 Docker 服务已经启动。

```
systemctl status docker
```

2.2 立即启动 Docker 服务：

如果没有启动

```
# 启动

systemctl start docker

# 设置开机自动启动

systemctl enable docker
```

2.3 验证是否成功

如果你看到 Active: active (running) 变成了绿色，就说明引擎已经发动，你可以继续去构建你的镜像了。

```
systemctl status docker
```

3 编写 Dockerfile

3.1 创建并编辑文件

Dockerfile 是一个没有后缀名的文本文件

```
vi Dockerfile
```

3.2 以创建一个简单的 Python 环境为例

Docker 的核心价值——环境隔离，不需要在宿主机安装 Python 也可以

```
# 创建 requirements.txt (哪怕现在没有第三方库，你也需要创建一个空文件或者填入一个常用的库)
```

```
echo "requests" > requirements.txt
```

```
-----  
下面是编写Dockerfile
```

```
# 使用官方的轻量级 Python 镜像作为基础
```

```
FROM python:3.9-slim
```

```
# 先复制依赖文件 (利用缓存)
```

```
COPY requirements.txt .
```

```
# 安装依赖
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# 再复制代码
```

```
COPY . .
```

```
# 启动命令
```

```
CMD ["python", "main.py"]
```

4 构建 (Build) 镜像

4.1 执行构建命令

在 Dockerfile 所在的目录下执行构建命令

命令格式: `docker build -t 镜像名称: 标签.`

注意: 末尾的 `.` 代表当前目录, 不能省略。

```
docker build -t my-python-app:v1 .
```

```
# 假设你的 project 文件夹在 root 目录下
```

```
cd /root/project

# 测试用的Python文件

echo "print('Docker build success! Hello from Python.')" > main.py

# 构建镜像

docker build -t my-python-app:v1 .

# 运行你的镜像

docker run --rm my-python-app:v1

如果成功应该是
Docker build success! Hello from Python.

# 查看镜像列表
docker images

# 查看镜像的详细层级
docker history my-python-app:v1
```

5 遇到问题与解决

1. **问题：**权限不足。
2. **解决：**使用 `sudo` 提升权限。